1 Input Validation: The Basics (Manual Validation)

Sabai vanda suru ma extra library use nagari Express ma manual validation kasari hunchha herdum. Yasle underlying concept bujhna sajilo banauchha.

Kunai `POST /register` route ma user le `email` ra `password` pathauda hami basic checks garchham:

JavaScript

```
const express = require('express');
const app = express();
app.use(express.json());

app.post('/register', (req, res) => {
    const { email, password } = req.body;

    // 1. Required Fields Check
    if (!email || !password) {
        return res.status(400).json({ error: "Email and password are
required." });
    }

    // 2. Format Validation (Basic Regex for Email)
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
        return res.status(400).json({ error: "Invalid email format." });
    }

    // 3. Custom Rule (Password length)
    if (password.length < 6) {
        return res.status(400).json({ error: "Password must be at least 6
characters long." });
    }

    res.status(200).json({ message: "User registered successfully!" });
});
```

Problem with Manual Validation: Code dherai run-time generic (if-else boilerplate) hunchha. Harek route ma yo garda code heavy ra repetitive hunchha. Tyai vayera hami ecosystem libraries use garchham.

2 2. Joi (Schema-based Object Validation)

Joi le runtime object ko schema validation garchha. Yo broad, declarative chha ra clean javascript objects validation ko lagi dangerous tools madhye ek ho.

3 Basic Implementation

JavaScript

```
const Joi = require('joi');

// Schema define gareko
const registerSchema = Joi.object({
    username: Joi.string().alphanum().min(3).max(30).required(),

    // Custom Error Message handle gareko .messages() batta
    email: Joi.string().email().required().messages({
        'string.email': 'Daju, email mildena! Please provide a valid email.',
        'any.required': 'Email pathauna mandatory chha.'
    }),

    password: Joi.string().pattern(new RegExp('^[a-zA-Z0-
9]{3,30}$')).required(),
    age: Joi.number().integer().min(18).max(100) // Optional clean setup
});
```

4 Express Middleware Setup (Advanced/Production Way)

Joi body validate garna Express middleware banayera check garnu best practice ho:

JavaScript

```
const validateBody = (schema) => {
    return (req, res, next) => {
        const { error, value } = schema.validate(req.body, { abortEarly:
false }); // abortEarly: false le sabai errors ekai choti dekhaucha

        if (error) {
            const errorMessages = error.details.map(err => err.message);
            return res.status(400).json({ errors: errorMessages });
        }

        // Sanitized asset overwrite garne
        req.body = value;
        next();
    };
};

// Route Implementation
app.post('/user', validateBody(registerSchema), (req, res) => {
    res.send("User Validated and Saved!");
});
```

5 3. express-validator (Middleware-driven Validation)

express-validator chain Express wrapper frameworks (validator.js mathi baneko) ko lagi tailored ho. Yasle req scope (body, query, params) lai direct router interface mai string handle garcha.

6 Basics to Advanced flow

JavaScript

```
const { body, query, validationResult } = require('express-validator');

app.post('/profile-update', [
    // 1. Required field & format validation
    body('username')
        .trim() // Sanitization: space falcha
        .notEmpty().withMessage('Username khali huna hudaina')
        .isLength({ min: 5 }).withMessage('Username minimum 5 chars
chaincha'),

    body('email')
        .isEmail().withMessage('Valid email chaincha')
        .normalizeEmail(), // Sanitization: lowercasing, ignoring dots in
gmail etc.

    // 2. Custom Validation (e.g., checking if email exists in Database)
    body('email').custom(async (value) => {
        const user = await req.db.findUserByEmail(value); // Hypothetical DB
check
        if (user) {
            throw new Error('Yo email registers vaisakeko cha');
        }
        return true;
    }),

    // 3. Sanitization Layer
    body('bio')
        .escape() // HTML tags primitive components swap garxa (XSS
protection)

], (req, res) => {
    // Error extraction
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
        return res.status(400).json({ errors: errors.array() });
    }

    // Validated response processing
    res.send("Profile sanitized & updated safely!");
});
```

7 4. Summary Table: Joi vs express-validator

Tapaile project ma kun use garne code context anusar select garna saknu hunchha:

Feature	Joi	express-validator
Approach	Schema-based (Object parsing)	Middleware-based (Request streaming)
Context	Independent (Any JS project)	Tied heavily with Express apps
Sanitization	Limits type transformation	Heavily supports (Trimming, Escaping, etc.)
DB handling	Dynamic inject garne, halka garo	Simple `.custom()` promises hooks

8 5. Advanced Architecture: Global Error Handler integration

Production-grade code ma validation errors direct route handler vitra handle garidena. Tyo error lai next middleware parameter pass garera **Centralized Global Error Middleware** bata formatting standard milainchha.

JavaScript

```
// custom error class
class AppError extends Error {
    constructor(message, statusCode) {
        super(message);
        this.statusCode = statusCode;
        this.isOperational = true;
        Error.captureStackTrace(this, this.constructor);
    }
}

// Global Validation Error Mapper Middleware
app.use((err, req, res, next) => {
    err.statusCode = err.statusCode || 500;

    // Format standardized responses
    res.status(err.statusCode).json({
        status: 'fail',
```

```
        message: err.message,
        stack: process.env.NODE_ENV === 'development' ? err.stack : undefined
    });
});
```